Flood Intelligence System

Project Architecture & Execution Overview

Purpose. Convert fragmented New Zealand flood reporting into a structured, evidence-linked Event -> Incident -> Impact dataset that can be explored through an interactive dashboard and used as a foundation for impact-based flood intelligence research.

Source articles	Verification pairs	Verified articles	Events	Incidents	Impacts
4,742	9,268	1,029	34	176	358

1. End-to-end architecture

1. News collection

DigitalNZ/news records are normalised and stored in **news_articles**.

↓

2. Broad candidate retrieval

The event catalogue and article text/metadata generate high-recall article-event candidates.

↓

3. LLM event verification

Each candidate is checked for temporal, geographic and disaster-context consistency.

↓

4. Primary assignment

Accepted high-confidence matches are resolved so one article has one primary event assignment.

↓

5. Grouped extraction

Verified articles are grouped by event and transformed into structured events, incidents, impacts, locations and evidence.

↓

6. Persistence

Results are stored in PostgreSQL and exported to CSV for lightweight deployment.

↓

7. Dashboard + RAG

Dash/Plotly provides maps, filters and charts; the event-scoped assistant answers using selected-event evidence only.

2. Why the pipeline is split into verification and extraction

The key design change is that candidate retrieval is **not treated as ground truth**. Retrieval is intentionally broad to protect recall, while the LLM verification stage decides whether an article genuinely describes the target real-world event. Only verified primary articles can enter grouped extraction.

- **Retrieval:** inexpensive and recall-oriented; may return noisy or duplicate candidates.
- **Verification:** tests event identity using time, place and disaster context, with a confidence threshold.
- **Extraction:** consolidates multiple verified articles into an event-level representation while retaining supporting article IDs and evidence text.
- **Auditability:** rejected and lower-ranked article-event decisions remain available in the validation table.

3. Core data model

Table	Role
news_articles	Raw/normalised source reporting.
flood_article_event_validation	Complete verification audit trail and confidence signals.
flood_event_articles	Articles accepted into a final flood event.
flood_events	Canonical event summary, date range, region and evidence.
flood_incidents	Distinct occurrences inside an event, with location/date/evidence.
flood_impacts	Consequences linked to incidents and events, with evidence and optional impact location.

4. Typical execution sequence

```
python flood_event_verified_pipeline.py prepare-verify
python flood_event_verified_pipeline.py process-all-verify-parts
python flood_event_verified_pipeline.py combine-verify
python flood_event_verified_pipeline.py import-verify
python flood_event_verified_pipeline.py prepare-extract
python flood_event_verified_pipeline.py submit-extract
python flood_event_verified_pipeline.py status --batch-id
python flood_event_verified_pipeline.py download-extract --batch-id
python flood_event_verified_pipeline.py import-extract
```

The pipeline also exposes **run-all** and command-specific help. Batch output files are written to the configured verification/extraction output directory.

5. Dashboard execution

Database-backed mode: `final_dashboard_verified_updated.py` connects directly to the structured PostgreSQL tables. This is useful during research and curation.

CSV-backed mode: `final_dashboard_csv.py` reads exported `flood_events`, `flood_event_articles`, `flood_incidents` and `flood_impacts` CSV files. This avoids hosting PostgreSQL and is appropriate for demonstrations and Vercel deployment.

- Dash filters by event, year, region, impact category and map readiness.
- Plotly shows impact hotspots, categories, regional concentration and annual patterns.
- Source article links expose provenance for manual review.
- The RAG assistant builds context from the selected event and its evidence, then calls the configured OpenAI model.

6. Environment and execution dependencies

Variable	Used for
OPENAI_API_KEY / OPENAI_MODEL	Verification, extraction and dashboard assistant.
DIGITALNZ_API_KEY	DigitalNZ collection.
SOURCE_DB_*	Read-only source database containing collected articles.
TARGET_DB_*	Structured flood-intelligence target database.
EVENT_EXCEL_PATH	Predefined event catalogue consumed by the verified pipeline.

7. Research boundary and next direction

This repository is a **research prototype**, not an operational flood-warning service. News coverage can be incomplete or biased, LLM extraction can make errors, and fallback map locations may be approximate. The next research step is to connect the historical Event -> Incident -> Impact knowledge base with hydrological, rainfall, forecast and exposure information so the system can contribute to impact-based flood warning/forecasting research.